

UNIDAD 8: APLICACIONES WEB CON SPRING, MVC Y THYMELEAF

Índice

UNIDAD 8: APLICACIONES WEB CON SPRING, MVC Y THYMELEAF	1
1. ¿Qué es Thymeleaf?	2
2. Creación de un “New Spring Starter Project Spring Boot”	6
a) Configuración	6
b) Creando las clases necesarias.....	12
c) Creación del controlador.....	12
d) Uso de Lombok	15
e) Creación de la vista	16
3. Primeros pasos con Thymeleaf	23
4. Configuración de Thymeleaf	25

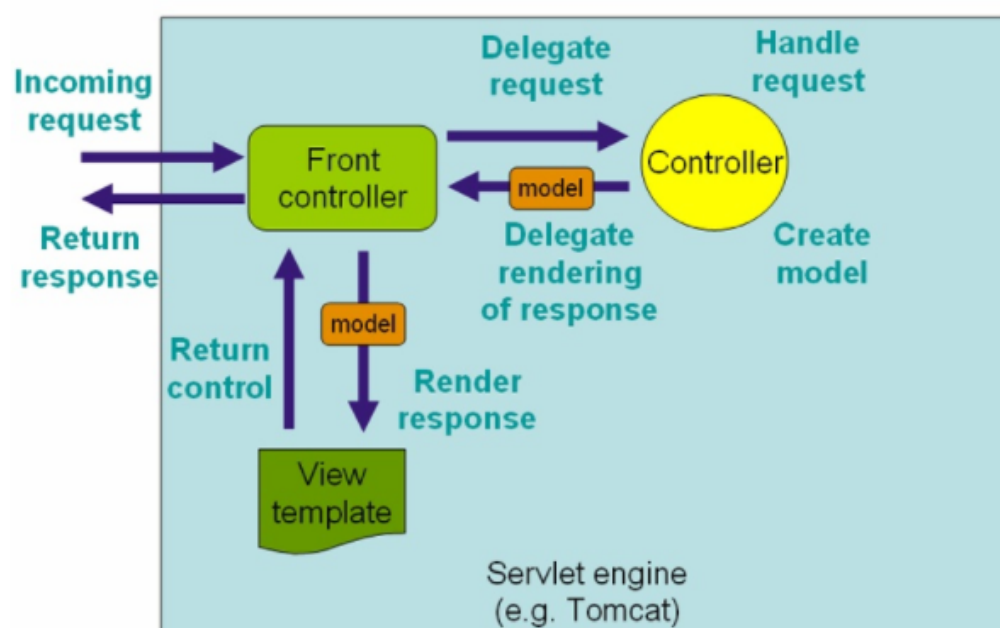


1. ¿Qué es Thymeleaf?

Veamos cómo se procesa una petición en Spring MVC:

Procesamiento de una petición en Spring MVC

A continuación se describe el flujo de procesamiento típico para una petición HTTP en Spring MVC. Esta explicación está simplificada y no tiene en cuenta ciertos elementos que comentaremos posteriormente. Spring es una implementación del patrón de diseño "front controller", que también implementan otros frameworks MVC, como por ejemplo, el clásico Struts.



- Todas las peticiones HTTP se canalizan a través del *front controller*. En casi todos los frameworks MVC que siguen este patrón, el *front controller* no es más que un servlet cuya implementación es propia del framework. En el caso de Spring, la clase `DispatcherServlet`.
- El *front controller* averigua, normalmente a partir de la URL, a qué *Controller* hay que llamar para servir la petición. Para esto se usa un `HandlerMapping`.
- Se llama al *Controller*, que ejecuta la lógica de negocio, obtiene los resultados y los devuelve al servlet, encapsulados en un objeto del tipo `Model`. Además se devolverá el nombre lógico de la vista a mostrar (normalmente devolviendo un `String`, como en JSF).
- Un `ViewResolver` se encarga de averiguar el nombre físico de la vista que se corresponde con el nombre lógico del paso anterior.
- Finalmente, el *front controller* (el `DispatcherServlet`) redirige la petición hacia la vista, que muestra los resultados de la operación realizada.

En realidad, el procesamiento es más complejo. Nos hemos saltado algunos pasos en aras de una mayor claridad. Por ejemplo, en Spring se pueden usar interceptores, que son como los filtros del API de servlets, pero adaptados a Spring MVC. Estos interceptores pueden pre y postprocesar la petición alrededor de la ejecución del *Controller*. No obstante, todas estas cuestiones deben quedar por fuerza fuera de una breve introducción a Spring MVC como la de estas páginas.

Thymeleaf es un motor de plantillas para aplicaciones web.

* ¿Qué es un motor de plantillas?

La idea general consiste en poder tener un HTML plano, no muy complejo y dentro, tener funciones que nos carguen los datos enviados desde el servidor (backend), por tanto, nos permite no estar retocando constantemente el HTML. Además, diseñador y programador pueden estar trabajando simultáneamente en el mismo proyecto.

Cada vez más en el front-end, consumimos lo que se llaman "rest-api", que te ofrecen datos que vienen del back-end en formato de texto, por ejemplo, en formato JSON. Lo que ofrecen estas librerías de plantillas es pasar rápidamente de esos datos a pedazos de código HTML que puedes insertar cómodamente en la página.

Thymeleaf soporta o puede generar plantillas de estos seis tipos:

- HTML
- XML
- TEXT
- JAVASCRIPT
- CSS
- RAW

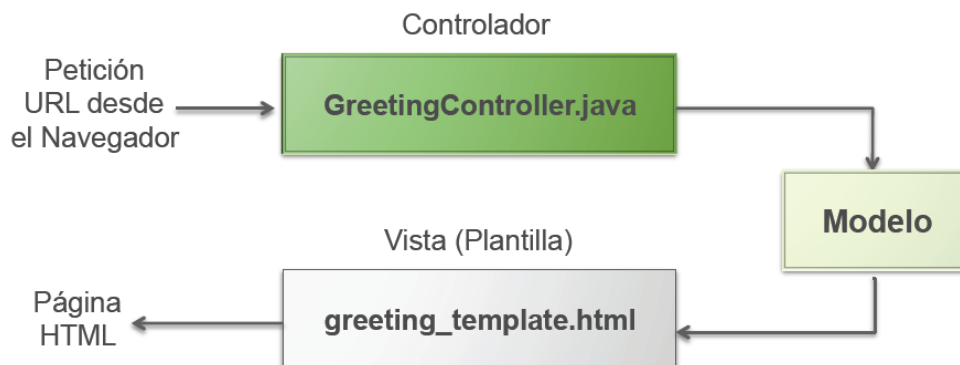
Thymeleaf permite realizar la maquetación HTML sin necesidad de que intervenga el servidor.

APLICACIONES WEB CON SPRING MVC Y THYMELEAF

1. Spring MVC

Introducción

- El modelo vista controlador (MVC) separa los datos y la lógica de negocio de una aplicación de la interfaz de usuario.
- El esquema del patrón MVC en Spring es:



<http://spring.io/guides/gs/serving-web-content/>

APLICACIONES WEB CON SPRING MVC Y THYMELEAF

1. Spring MVC

Controladores

- Los controladores (*Controllers*) son clases Java encargadas de atender las peticiones web.
- Procesan los datos que llegan en la petición (parámetros).
- Hacen peticiones a la base de datos, usan diversos servicios, etc...
- Definen la información que será visualizada en la página web (el modelo).
- Determinan que vista será la encargada de generar la página HTML.

APLICACIONES WEB CON SPRING MVC Y THYMELEAF

1. Spring MVC

Vistas

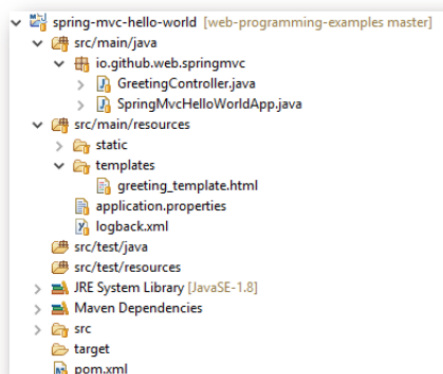
- Las vistas en Spring MVC se implementan como plantillas HTML.
- Generan HTML partiendo de una plantilla y la información que viene del controlador (modelo).
- Existen diversas tecnologías de plantillas que se pueden usar con Spring MVC: JSP, Thymeleaf, FreeMarker, etc...
- Nosotros usaremos **Thymeleaf**.

APLICACIONES WEB CON SPRING MVC Y THYMELEAF

1. Spring MVC

Ejemplo

- Estructura de la aplicación vista desde Eclipse:



Creación de un proyecto Spring con Thymeleaf. Mi segundo "Hola mundo".

En esta imagen vemos el aspecto de un archivo html con código y cómo queda cuando es renderizado, es decir, cuando se pasa a su representación gráfica final.

```
<table id="myTableId" dt:table="true">
  <thead>
    <tr>
      <th>Id</th>
      <th>Firstname</th>
      <th>Lastname</th>
      <th>Street</th>
      <th>Mail</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="person : ${persons}">
      <td th:text="${person.id}">1</td>
      <td th:text="${person.firstname}">John</td>
      <td th:text="${person.lastname}">Doe</td>
      <td th:text="${person.address.street1}">Nobody knows !</td>
      <td th:text="${person.mail}">johndoe.com</td>
    </tr>
  </tbody>
</table>
```

Renderizado

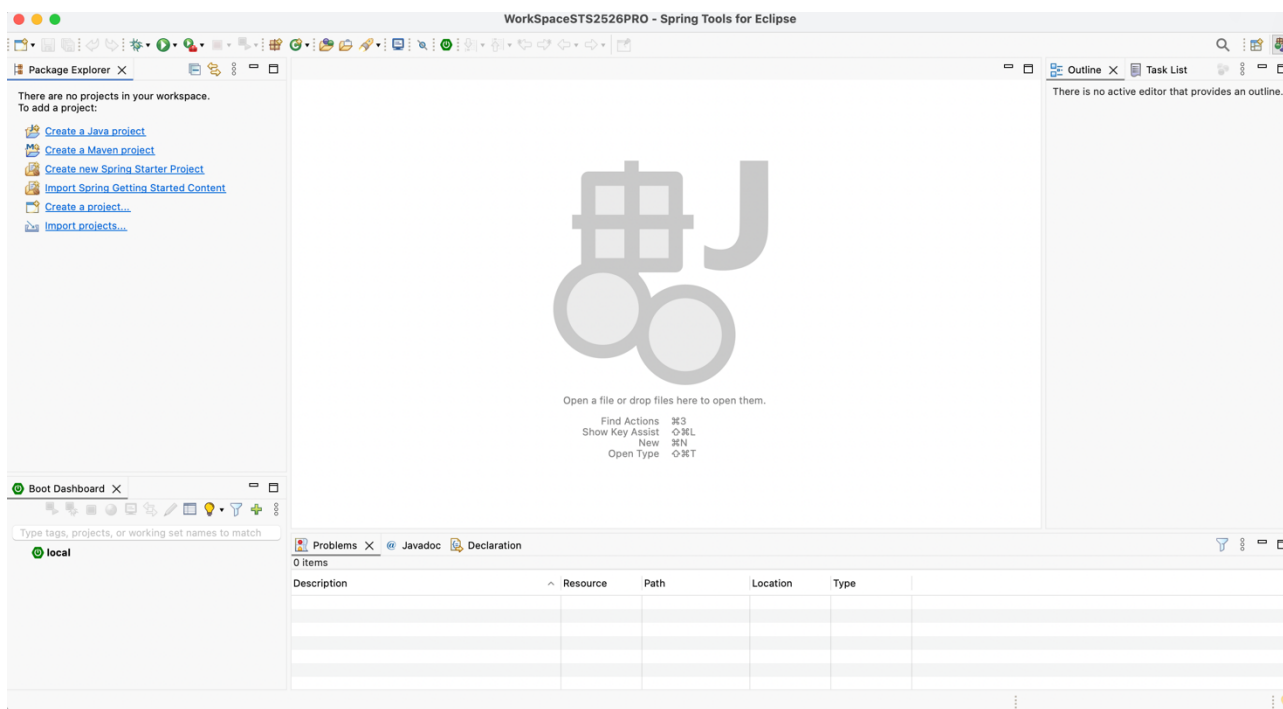
As a result, you'll have the following full-featured HTML table:

Show 10 entries

Id	Firstname	Lastname	Street	Mail
1	Salma	Maldonado	Ap #351-1812 Eu Ave	venenatis@duisulpat.com
2	Vanna	Salas	947-3009 Feugiat St	lobortum fermentum metus@ante.ca
3	Nedra	Roadies	1389 Aliquam St	Duis curabitur diam@duisulpat.com
4	Reina	Gonzalez	P.O. Box 374, 9474 Lacrima Street	tut lacus non magna@duis.edu
5	Calista	Hill	8968 Eu Road	amet Etiam imperdiet@duisulpat.com
6	China	Wilder	317-6083 Ligula St	lacus vestibulum@duisulpat.com
7	Oliver	Rivas	6480 Sed Ave	velit@duisulpat.com
8	Lila	Munoz	8436 Verna Ave	Mauris ac lacinia@duisulpat.com
9	Yeni	Yonges	P.O. Box 722, 9079 Eu St	duis@duisulpat.com
10	Lee	Graves	211-4391 Octum Road	velit@duisulpat.com

Showing 1 to 10 of 100 entries

Una vez instalado el Spring Tool Suite, al abrir nos pedirá una carpeta como espacio de trabajo y la primera vista que aparece será:



2. Creación de un “New Spring Starter Project Spring Boot”

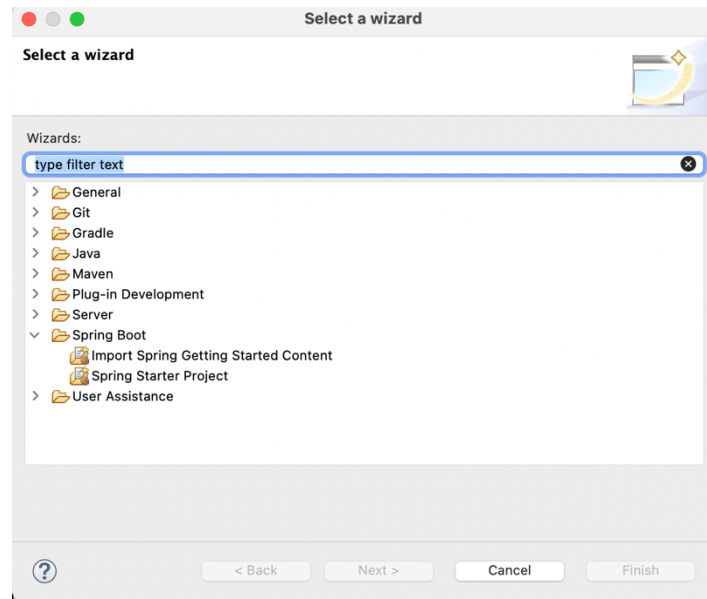
Nota: Solo hay que tener descargado antes el IDE Spring Tool Suit.

Veámoslo paso a paso.

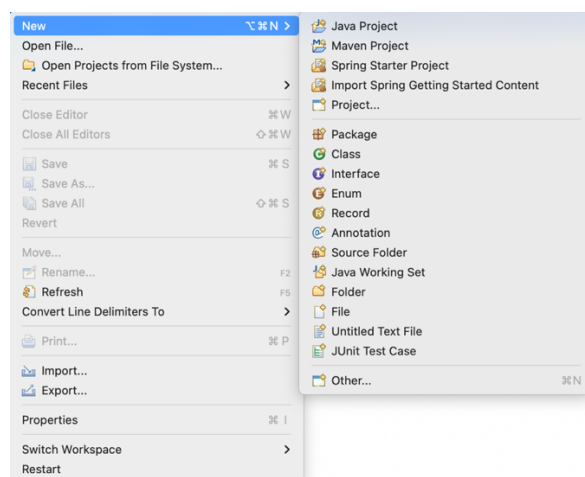
a) Configuración

Paso 1: New/Spring Starter Project

La primera vez no aparecerá en los recientes, por lo que estará en:



Posteriormente, ya aparecerá en recientes:



En la ventana que aparece hay que modificar, elegir y nombrar las siguientes cosas (si no se dice nada sobre ellas aquí se dejan tal como salen por defecto):

- Name: debemos cumplir lo de cualquier proyecto java, mayúsculas, minúsculas...

Ejemplo: ProyectoEj01HolaMundo

- Type: elegimos Maven
- Java Version: 21
- Group: todo minúscula. Nombre del dominio del colegio (imaginario, no es real) pero siempre pondremos: com.salesianostriana.dam

Es la base o raíz de las rutas que usaremos en el proyecto para las distintas carpetas y archivos.

- Artífacto: Mismo nombre del proyecto, pero todo en minúscula (aunque no tendría que ser en minúscula obligatoriamente, nosotros lo haremos así).

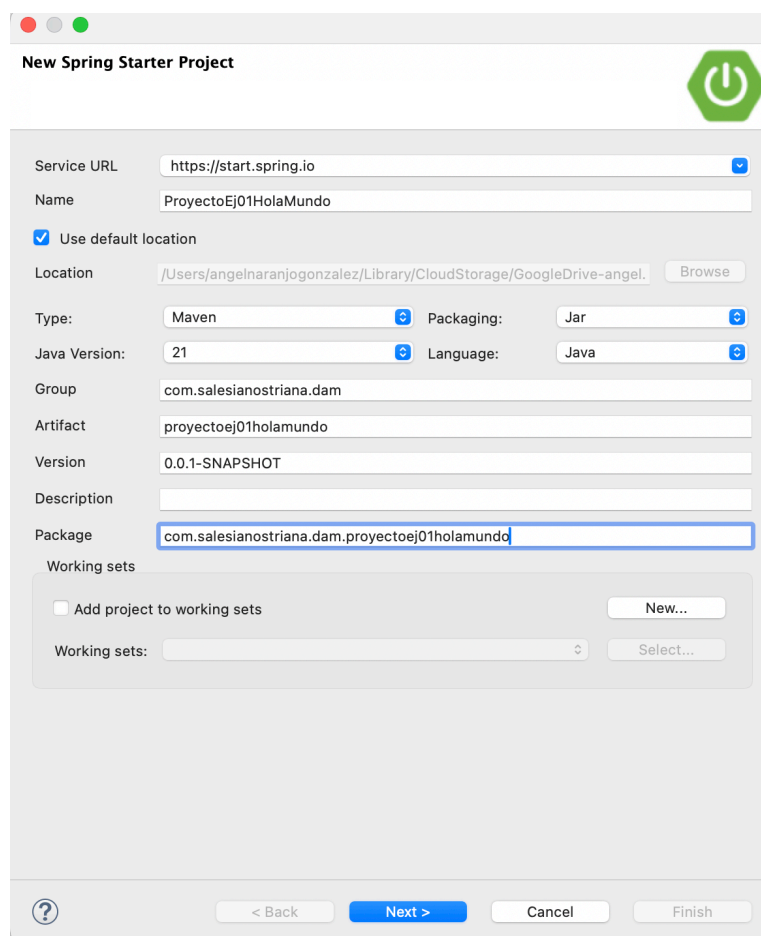
Básicamente un artefacto es un bloque de código reutilizable. Si alguien quiere saber más dejo un enlace en la plataforma.

Ejemplo: ud8e01holamundo

- Paquete: Composición del nombre del grupo y del artefacto, todo en minúscula. Por ejemplo:

com.salesianostriana.dam.proyecto01holamundo

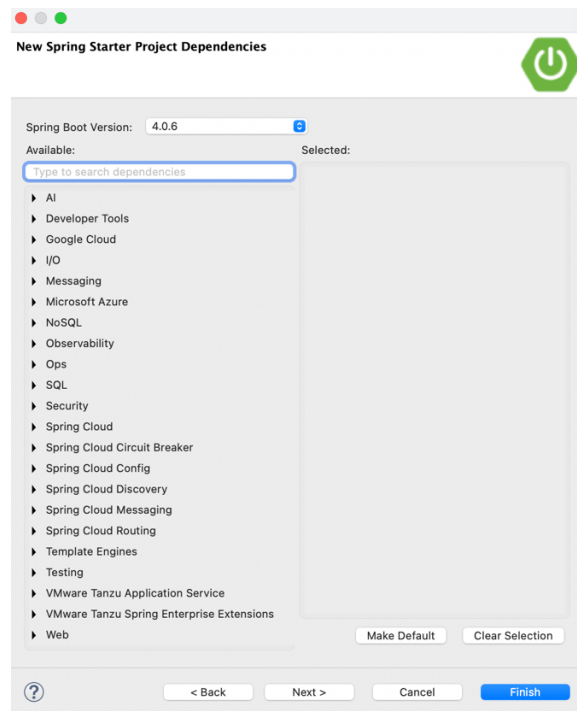
En nuestro ejemplo, podría quedar algo así:



- **Pinchamos en Next**

Aparece la siguiente página, dedicada a las dependencias. La primera vez no, pero después aparecerán las más recientes ya buscadas. También se podrán agregar después, pero mejor hacerlo todo antes de empezar.

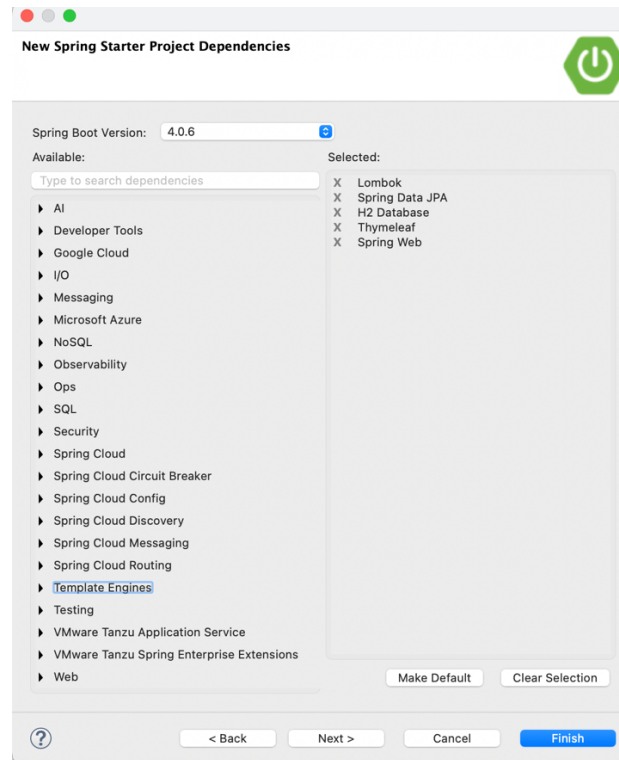
En primer lugar, elegimos la versión de Spring Boot Version. En nuestro caso, la última que recomiende el STS que no sea Snapshot (4.0.6)



Debemos marcar los tipos de dependencias que queremos incluir. Al menos de momento debemos marcar:

- En Web (Spring Web)
- En Template Engine (Thymeleaf)
- En SQL (H2 Database y Spring Data JPA)
- En Developer Tools (Lombok)
- Y todas aquellas que vayan haciéndose necesarias conforme nuestro proyecto vaya creciendo.

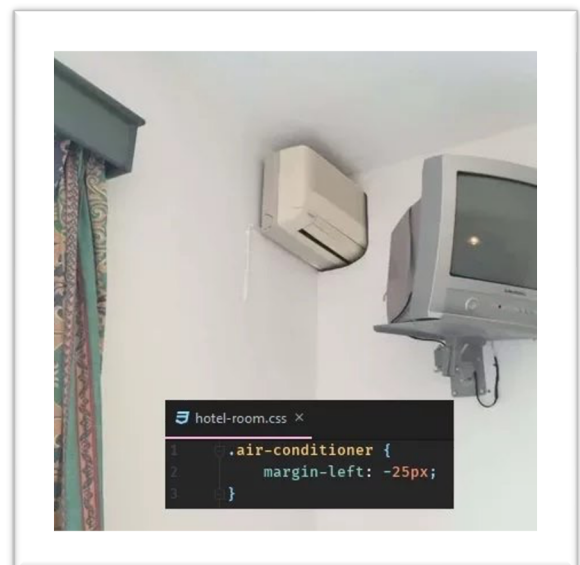
Debería quedar algo así:



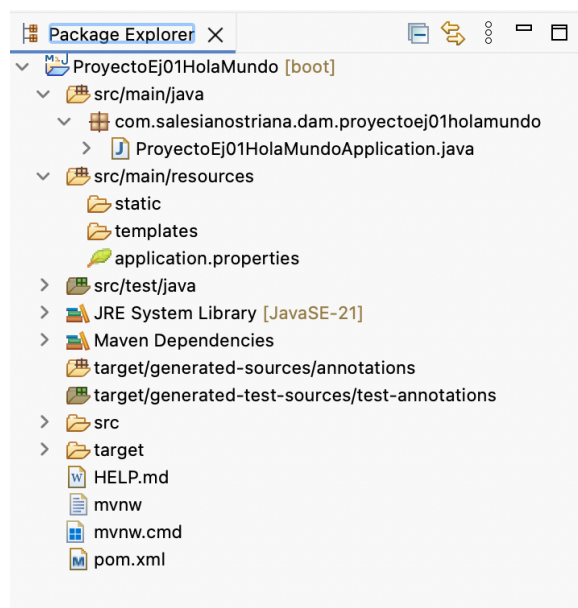
- **Pinchamos en Finish**

Paso 2: Nuestro proyecto comienza a crearse descargando e importando lo necesario.

Dependiendo de la conexión a internet y el ordenador, puede tardar unos segundos (no te asustes si aparece mensaje de error al principio) aparecerá nuestro proyecto. Puedes ver la barra de proceso abajo a la derecha.



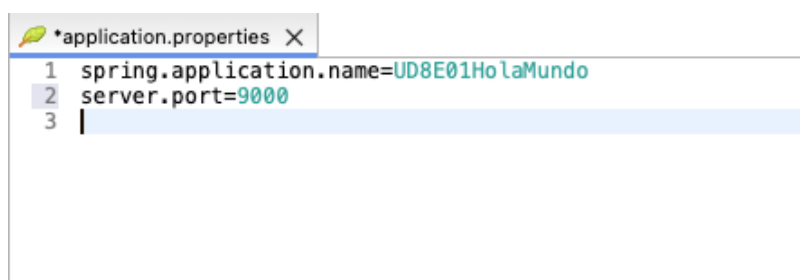
- Estructura de carpetas creada



- Cambio de puerto

Pinchamos en el archivo `application.properties` que está dentro de la carpeta `src/main/resources` y escribimos: `server.port=9000` (puede ser también el 8090 o cualquier otro pero debemos recordarlo para luego escribir dicho puerto en la ruta del navegador para ver la página) como se puede ver en la imagen y guardamos. Esto se hace por si el 8080 que es el puerto por defecto está ocupado.

Poned siempre 9000 para que todos usemos el mismo.



b) Creando las clases necesarias

Cuando se vayan escribiendo clases, STS nos pedirá importar los paquetes o clases necesarias para poder usar las anotaciones propias de Spring.

NOTA: Un atajo para importar todas las librerías necesarias directamente es **CTRL+SHIFT+"LETRA O"** en Windows (**COMMAND+SHIFT+LETRA O** en Mac)

Por otro lado, debemos observar que ya se ha creado la clase principal, llamada `ProyectoEj01HolaMundoApplication`, donde se encuentra el main y desde donde arrancará nuestra aplicación. Se verá la posibilidad de escribir en el main algún código más para poder ejecutar ciertas cosas de momento para probar nuestros ejemplos.

c) Creación del controlador

Debemos crear una clase dentro del paquete que ya tenemos creado en `src/main/java/com.salesianostriana.dam.proyectoelj01holamundo`.

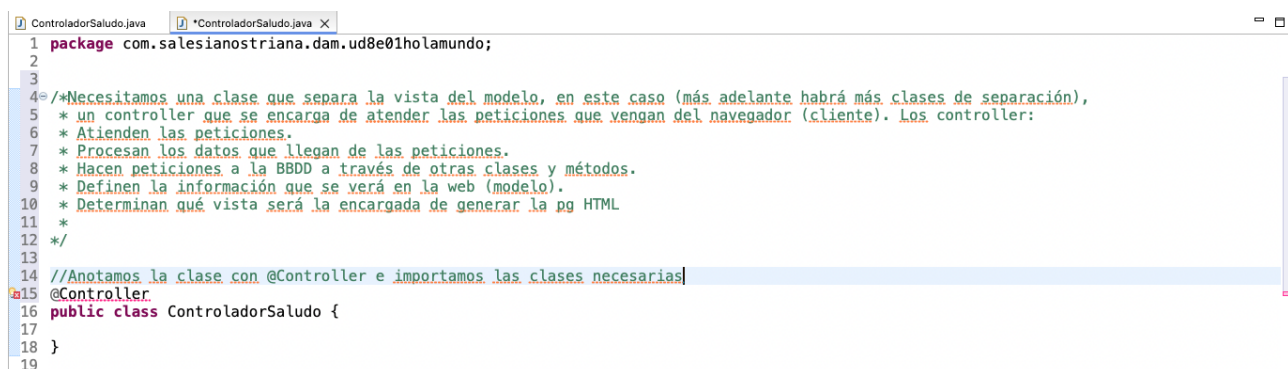
De momento a esta clase, la llamaremos `ControladorSaludo.java`

Un controlador es un método que atiende a peticiones, por lo que debe ir anotado (palabras con @ encima de su firma) con `@Controller`. Al poner esta, STS nos da un error porque la clase no está importada.

Ya hemos dicho que podemos ir importando paquetes mediante `control+shift+letra O`. También como siempre pinchando en el error y eligiendo las clases que nos propone el IDE o escribiendo nosotros mismos los import.

El controlador en esta ocasión es una clase muy simple. Spring, facilita mucho la creación de controladores, que atenderán las peticiones de nuestra aplicación antes de enviar los datos a la vista y, en principio, que no debe heredar de ninguna clase especial.

Te darás cuenta que hay un error de compilación ya que no hemos creado la clase `Persona`. Se explica esto un poco más adelante para aprovechar también el uso de la librería Lombok.



```

1 package com.salesianostriana.dam.ud8e01holamundo;
2
3
4 /*Necesitamos una clase que separa la vista del modelo, en este caso (más adelante habrá más clases de separación),
5 * un controller que se encarga de atender las peticiones que vengan del navegador (cliente). Los controller:
6 * Atienden las peticiones.
7 * Procesan los datos que llegan de las peticiones.
8 * Hacen peticiones a la BBDD a través de otras clases y métodos.
9 * Definen la información que se verá en la web (modelo).
10 * Determinan qué vista será la encargada de generar la pg HTML
11 *
12 */
13
14 //Anotamos la clase con @Controller e importamos las clases necesarias
15 @Controller
16 public class ControladorSaludo {
17
18 }
19

```

La clase completa sería:

```
package com.salesianostriana.dam.ud8e01holamundo;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

/* Necesitamos una clase que separa la vista del modelo, en este caso (más adelante
habrá más clases de separación),
 * un controller que se encarga de atender las peticiones que vengan del navegador
(cliente). Los controller:
 * Atienden las peticiones.
 * Procesan los datos que llegan de las peticiones.
 * Hacen peticiones a la BBDD a través de otras clases y métodos.
 * Definen la información que se verá en la web (modelo).
 * Determinan qué vista será la encargada de generar la pg HTML
 */

//Anotamos la clase con @Controller e importamos las clases necesarias
@Controller
public class ControladorSaludo {

    /* Ejemplo 1: primer controlador, llamado welcome
    *
    * "/" o /welcome" es el nombre del recurso que queremos mostrar al inicio, por
    * tanto la palabra que se debe escribir en la ruta que tendremos que escribir
    * en el navegador después de Localhost:9000 (o nada o welcome y posteriormente
    * se puede probar con un parámetro nombre)
    *
    * @GetMapping indica que el método justo de debajo será el que atenderá la
    * petición tipo Get (la petición es tipo Get solo hay un método y dos cuando es
    * tipo Post (formulario). Puede que en códigos antiguos os encontréis el uso
    * de RequestMapping en lugar de GetMapping.
    *
    * El @RequestParam se utiliza para pasar un parámetro a la petición tipo get, es
    decir, información o datos que
    * vamos a pasar en una petición tipo get (recuerda que no están pensadas para esto
    pero hay muchas veces
    * en las que llevan algún parámetro.
    * Si en la barra del navegador pasamos este parámetro con un valor aparecerá en el
    saludo, esto es,
    * si escribes en el navegador http://localhost:9000/?nombre=Luis aparecerá el saludo
    personalizado para Luis,
    * si no hay parámetro, el valor por defecto es Mundo
    */

    @GetMapping({ "/", "welcome" })
    public String welcome(@RequestParam(name = "nombre", required = false,
defaultValue = "Mundo") String nombre,
        Model model) {

        /*
        * El parámetro "nombre" es la palabra que usamos para darle un identificador a
        * la variable y debemos usar en la plantilla dentro de <p>Hello <span
        * th:text="${nombre}">Friend</span>!</p> Ojo porque no tener un criterio a la
```

```

    * hora de dar estos nombres a las variables hace que tengamos muchos errores
    * (usar mayúsculas o minúsculas, que el nombre no indique qué guarda, mezclas
    * varias palabras...
    */

    modelo.addAttribute("nombre", nombre);// Incluimos la información en el
modelo

    return "index";// Nombre de la plantilla que generará la página HTML (sin
extensión),
    // en nuestro caso estos html deben estar dentro de la carpeta
    // src/main/resources/templates
    // y deben llamarse igual que el String que devuelve el método,
    // index.html Ojo con las mayúsculas y minúsculas del nombre
    // de la plantilla porque son tenidas en cuenta
}

/*
 * Ejemplo 2: Segundo controlador llamado welcome2
 * Atiende a la petición /saludo2 escrita en el @GetMapping
 *
 * Se puede ver cómo se "pasa o carga información" al
 * model dentro del método mediante el método addAttribute
 * como un objeto de la clase Persona, con valores de sus dos
 * atributos. Más adelante iremos viendo de dónde sacamos esa "información" que
 * cambiar, como aquí la new Persona creada directamente en el método
addAttribute.
*/

@GetMapping("/saludo2")
public String welcome2(Model model) {

    modelo.addAttribute("persona", new Persona("Ángel", "Naranjo González"));
    return "SaludoPersonalizado";
}

/*
 * @GetMapping es una variante de requestMapping, más "nueva" que se utiliza
 * como atajo, ya que basta con el nombre del recurso, mientras que
 * con @RequestMapping, en general, tenemos que ir diciendo el tipo de petición
 * que se está atendiendo, es decir, necesita que le indiquemos el value
 * (nombre el recurso, por ejemplo, /saludo3) y el método que se usa para la
 * petición, en nuestro caso, tendríamos que escribir:
 * @RequestMapping (value="/saludo3", method=RequestMethod.GET)
 * (también existe RequestMethod.POST)
 * En general, se usa por simplificar el código
 * Se puede ver un ejemplo en:
 * https://www.arquitecturajava.com/spring-getmapping-postmapping-etc/
 *
 * Nosotros usaremos siempre GetMapping
*/

/* Ejemplo 3: Tercer controlador llamado welcome3
 * Atiende a la petición get "/saludo3"
 *
 * */

@GetMapping ("/saludo3")
public String welcome3(Model model) {

    modelo.addAttribute("saludo", "Hola Mundo");
    modelo.addAttribute("mensaje", "¡Se me está haciendo largo el proyecto final!");
    modelo.addAttribute("url", "https:// triana.salesianos.edu");

```



```
        return "SaludoYEnlace";  
    }  
}
```

En primer lugar, llama la atención la simplicidad de la clase, es Java en estado puro, anotada como hemos dicho con `@Controller`

La clase tiene un varios métodos, cuya *firma* es `public String nombreMetodo(Model model)`:

- Como decíamos más arriba, el tipo de retorno es `String`, ya que devolverá **el nombre** de una vista en formato cadena y sin la extensión `html`.
- Como argumento, recibe un elemento de tipo `Model`. Será el "contenedor" que nos permita añadir datos que serán "transportados" a la vista. Iremos viendo más formas de añadir cosas al `model`, de momento, solo con `addAttribute`.
- Además, están anotados con `@GetMapping("/saludo3")`. De esta forma indicamos que cualquier petición hacia la url `http://máquina:puerto/HelloWorldSpringMVC/saludo3` será procesada por este método.

d) Uso de Lombok

Lombok es una biblioteca de Java que reduce drásticamente el código repetitivo (*boilerplate*) como getters, setters, constructores y `toString()` mediante anotaciones en tiempo de compilación. Al eliminar la necesidad de escribir métodos repetitivos, mejora la legibilidad y agiliza el desarrollo, sin penalizar el rendimiento en tiempo de ejecución.

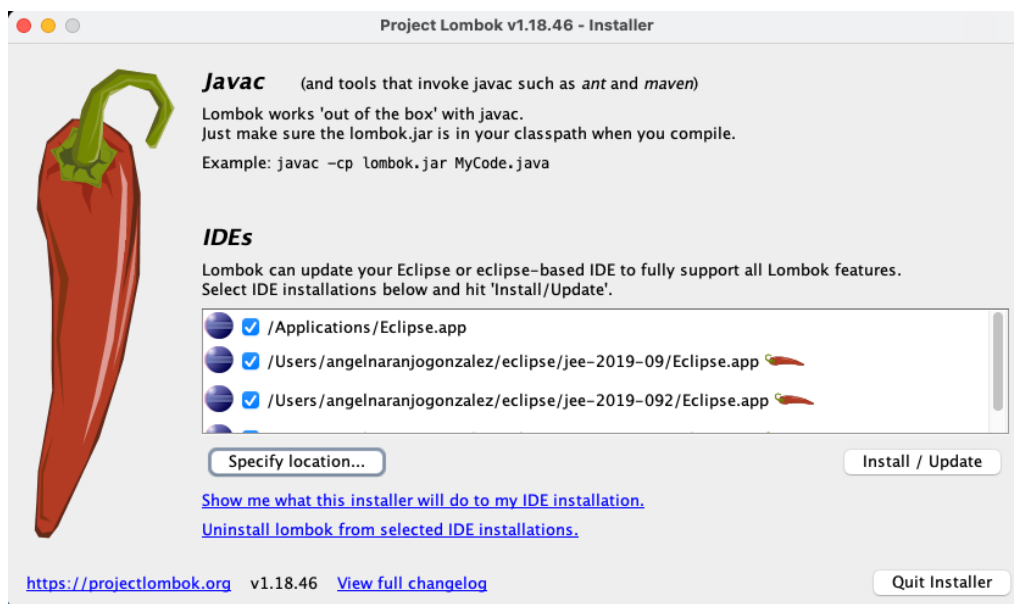
Genera el código automáticamente al compilar (no en tiempo de ejecución) insertándolo en el bytecode.

- **Anotaciones comunes de Lombok son:**

- `@Getter` / `@Setter`: Genera métodos de acceso.
- `@ToString`: Implementa el método `toString()`.
- `@NoArgsConstructor` / `@AllArgsConstructor`: Genera constructores.
- `@Data`:
Empaqueta `@Getter`, `@Setter`, `@ToString`, `@EqualsAndHashCode` y `@RequiresArgsConstructor` en una sola.
- `@Builder`: Implementa el patrón de diseño Builder.

Se añade como dependencia en el `pom.xml` (cuando se usa Maven) o `build.gradle` (si se usa Gradle) y requiere previamente un IDE, como puede ser IntelliJ, Eclipse, etc.

En primer lugar, debemos descargar Lombok, descomprimir y ejecutar. Llegamos a:



En esta imagen vemos cómo Lombok detecta Eclipse, pero en nuestro caso queremos utilizarlo con STS, así que lo buscamos pinchando en Specify location antes de hacer click en Install/Update.

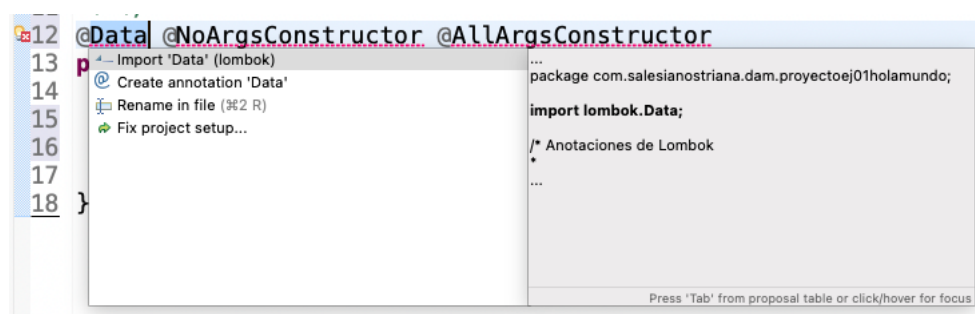
En windows suele estar en: /Applications

En Mac, en: /Applications/SpringToolSuite4.app/Contents/Eclipse/SpringToolSuite4.ini.

La instalación es muy rápida y enseguida tendremos la ventana de instalación exitosa. Debemos reiniciar nuestro IDE.

Al comenzar de nuevo, podemos añadir la clase Persona utilizando las anotaciones de Lombok.

Ten en cuenta que debemos importar lo necesario:

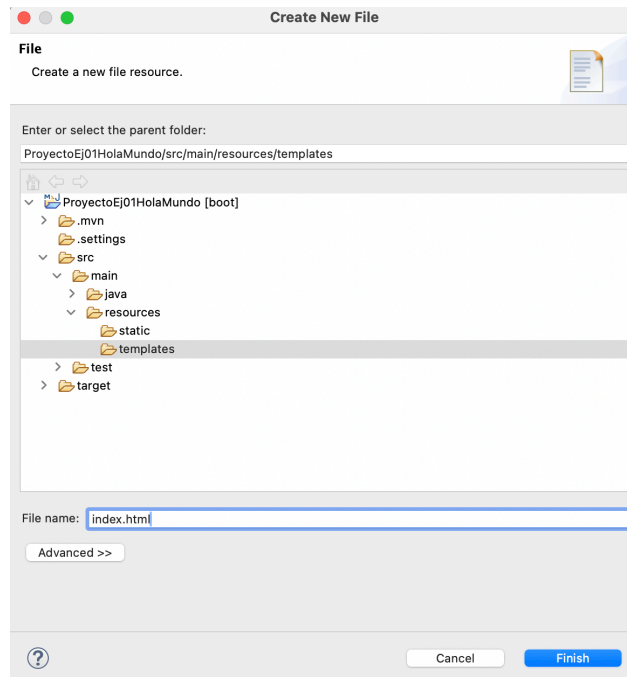


e) Creación de la vista

Para crear la vista, escribiremos nuestros archivos con código HTML en la carpeta "templates" dentro de la ruta de carpetas del proyecto src/main/resources.

Para ello basta crear un archivo tipo HTML File pinchando con el botón derecho en la carpeta templates, new y crear un archivo tipo File y añadir en el nombre la extensión .html. El nombre de este archivo debe ser el mismo que el String que devuelve el método controlador que atiende a la petición de welcome.

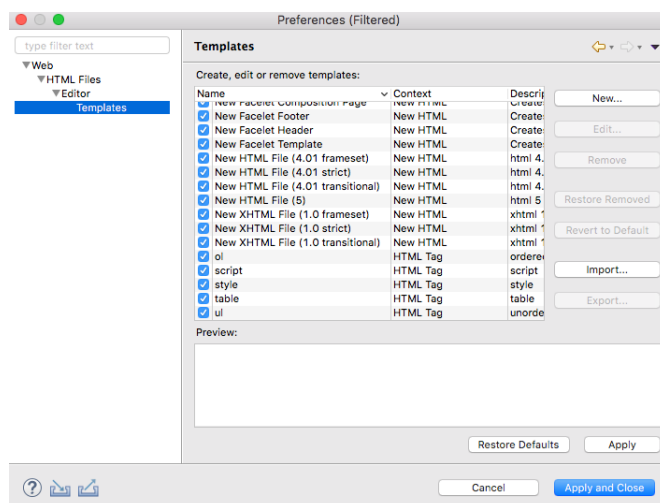
En nuestro caso, el primero que haremos será el index.html que es la página que se pinta al atender la petición del primer ejemplo.



NOTA: Si se instala el plugin adecuado, el propio STS ofrece la posibilidad de crear directamente un archivo tipo HTML (no File genérico) y con él podemos configurar algunos aspectos de nuestros archivos html. Ahora mismo no lo trae STS así que no lo haremos aquí, pero tiene más o menos este aspecto:



Se pueden realizar algunos cambios en la plantilla que por defecto ofrece STS. Para ello, pinchamos en el vínculo HTML Templates. Aparecerá la siguiente ventana:



Nosotros no instalaremos ese plugin. Escribimos el siguiente código en el archivo html. Nota: El siguiente código pertenece al archivo html llamado "index.html".

En la segunda línea, se define lo que se llama el "espacio de nombres" que avisa a html de que dentro se utilizarán etiquetas de Thymelaf, mediante "th:"

```
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1,
  shrink-to-fit=no">
  <title>Hello, world!</title>
</head>

<body>

  <h1>
    Hola <span th:text="${nombre}">mundo</span>
  </h1>
  <h2>Bienvenido al... Infierno</h2>

</body>

</html>
```

Si te fijas, la parte que irá cambiando según el parámetro nombre que le demos, lleva delante th: ya que es la que se refiere a Thymeleaf. Este parámetro debe llamarse igual que la variable usada en el método controller.

```
@GetMapping({ "/", "welcome" })
public String welcome(@RequestParam(name = "nombre", required = false,
defaultValue = "Mundo") String nombre, Model model) {

    model.addAttribute("nombre", nombre);
    return "index";
}
```

Además, encontramos que hemos usado *Expression Language*, una característica soportada por Thymeleaf que nos permite utilizar un lenguaje sencillo para usar un *JavaBean* dentro de una página HTML.

\${nombre} buscará dentro del contexto/scope (ámbito de vida de la petición) una variable llamada *nombre* (ahora, el nombre de la variable es lo que va entre comillas dobles dentro de los parámetros del método *addAttribute*, y mostrará su valor (que es el parámetro que va en segundo lugar, en marrón).

Si agregas un nombre como parámetro en la petición que se escribe en la web, aparecerá Hola y el nombre pasado.

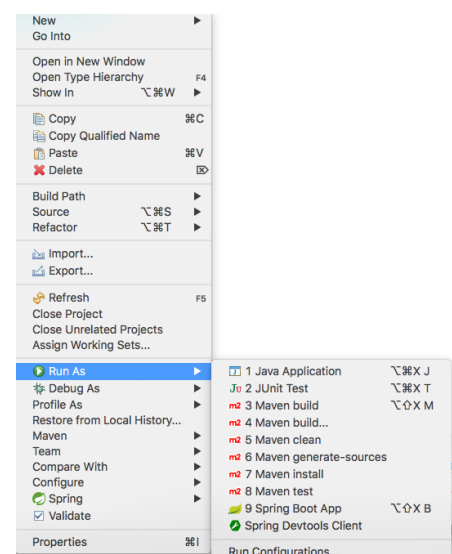
Por ejemplo: <http://localhost:9000/?nombre=Luis>

Para empezar a entender la parte de Thymeleaf, vamos a utilizar sus propios tutoriales, pero puedes ver que está relacionado con las etiquetas th: que aparecen en el html de este ejemplo.

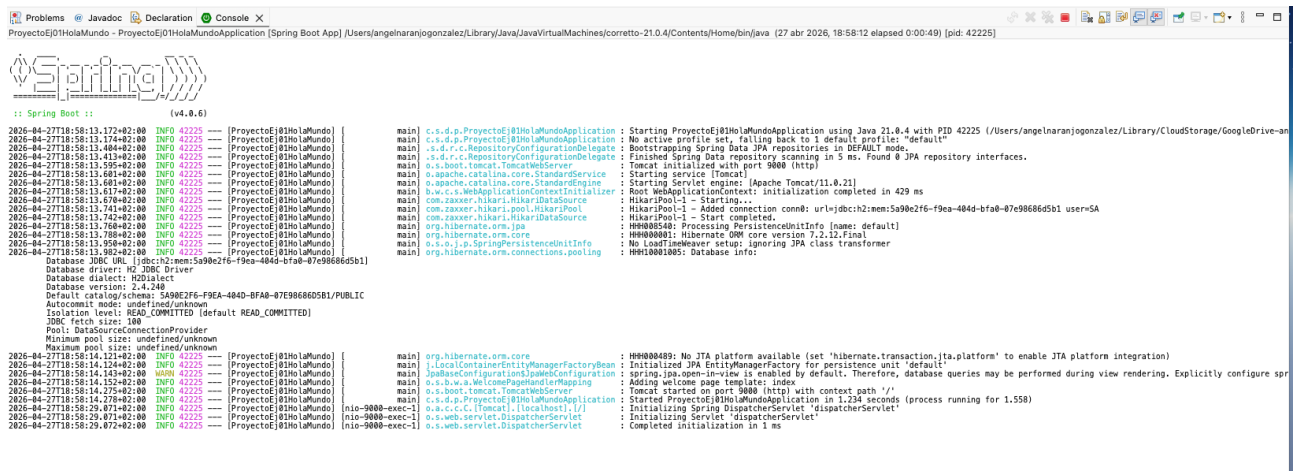
Paso 4: Ejecución

Una vez que tenemos definido nuestro proyecto, lo hemos configurado, y hemos añadido una vista y un controlador, vamos a pasar a ejecutarlo.

Para ello, tan solo tenemos que pulsar sobre el proyecto con el botón derecho > *Run As* > *Spring Boot App*.



Si todo va bien, debe aparecer en la consola algo como la siguiente imagen:



```

2026-04-27T18:58:13.172+02:00 INFO 42225 --- [ProjectE@HolaMundo] main c.s.d.p.ProjectE@HolaMundoApplication : Starting ProjectE@HolaMundoApplication using Java 21.0.4 with PID 42225 (/Users/angelnaranjogonzalez/Library/CloudStorage/GoogleDrive-angelnaranjogonzalez/Projects/ProjectE@HolaMundoApplication [Spring Boot App] /Users/angelnaranjogonzalez/Library/Java/JavaVirtualMachines/corretto-21.0.4/Contents/Home/bin/java (27 abr 2026, 18:58:12 elapsed 0:00:49) [pid: 42225])
2026-04-27T18:58:13.174+02:00 INFO 42225 --- [ProjectE@HolaMundo] main c.s.d.p.ProjectE@HolaMundoApplication : No active profile set, falling back to 1 default profile: "default"
2026-04-27T18:58:13.404+02:00 INFO 42225 --- [ProjectE@HolaMundo] main c.s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-04-27T18:58:13.413+02:00 INFO 42225 --- [ProjectE@HolaMundo] main c.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 5 ms. Found 0 JPA repository interfaces.
2026-04-27T18:58:13.595+02:00 INFO 42225 --- [ProjectE@HolaMundo] main o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 9000 (http)
2026-04-27T18:58:13.601+02:00 INFO 42225 --- [ProjectE@HolaMundo] main o.s.boot.tomcat.TomcatWebServer : Starting service (Tomcat)
2026-04-27T18:58:13.601+02:00 INFO 42225 --- [ProjectE@HolaMundo] main o.s.boot.tomcat.TomcatWebServer : Starting Servlet engine: (Apache Tomcat/11.0.21)
2026-04-27T18:58:13.617+02:00 INFO 42225 --- [ProjectE@HolaMundo] main o.s.b.w.c.WebApplicationContext : Root WebApplicationContext: initialization completed in 429 ms
2026-04-27T18:58:13.678+02:00 INFO 42225 --- [ProjectE@HolaMundo] main com.zaxxer.hikari.HikariPool : HikariPool-1 - Starting...
2026-04-27T18:58:13.742+02:00 INFO 42225 --- [ProjectE@HolaMundo] main com.zaxxer.hikari.HikariPool : HikariPool-1 - Start completed.
2026-04-27T18:58:13.788+02:00 INFO 42225 --- [ProjectE@HolaMundo] main org.hibernate.orm.jpa : HH000048: Processing PersistenceUnitInfo [name: default]
2026-04-27T18:58:13.908+02:00 INFO 42225 --- [ProjectE@HolaMundo] main org.hibernate.orm.jpa : HH000049: No JTA platform available (set 'hibernate.transaction.jta-platform' to enable JTA platform integration)
2026-04-27T18:58:13.908+02:00 INFO 42225 --- [ProjectE@HolaMundo] main org.hibernate.orm.jpa : HH000080: Hibernate ORM core version 7.2.12.Final
2026-04-27T18:58:13.908+02:00 INFO 42225 --- [ProjectE@HolaMundo] main org.hibernate.orm.connections.pooling : HH000105: Database info:
Database driver: H2 JDBC Driver
Database dialect: H2Dialect
Database version: 2.4.200
Default catalog/schema: SA90E2F6-F9EA-484D-BFA8-07E98686D5B1/PUBLIC
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED (default: READ_COMMITTED)
JDBC fetch size: 100
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-04-27T18:58:14.124+02:00 INFO 42225 --- [ProjectE@HolaMundo] main j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-04-27T18:58:14.143+02:00 INFO 42225 --- [ProjectE@HolaMundo] main j.LocalContainerEntityManagerFactoryBean : Spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this behavior
2026-04-27T18:58:14.275+02:00 INFO 42225 --- [ProjectE@HolaMundo] main c.s.d.p.ProjectE@HolaMundoApplication : Adding welcome page template: index
2026-04-27T18:58:14.278+02:00 INFO 42225 --- [ProjectE@HolaMundo] main c.s.d.p.ProjectE@HolaMundoApplication : Tomcat started on port 9000 (http) with context path '/'
2026-04-27T18:58:29.071+02:00 INFO 42225 --- [nio-9000-exec-1] o.s.c.c.t.Tomcat [localhost] [/] : Started ProjectE@HolaMundoApplication in 1.224 seconds (process running for 1.558)
2026-04-27T18:58:29.072+02:00 INFO 42225 --- [ProjectE@HolaMundo] main org.springframework.boot.web.servlet : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-04-27T18:58:29.072+02:00 INFO 42225 --- [ProjectE@HolaMundo] main org.springframework.boot.web.servlet : Completed initialization in 1 ms
  
```

Para ver nuestra aplicación, basta con ir a un navegador y escribir en la barra cualquiera de estas dos rutas:

localhost:9000/

localhost:9000/welcome

Si la escribimos en el navegador, veremos que aparece nuestra esperada vista.



Hola Mundo

Bienvenido al... Infierno

Cuando queramos parar la ejecución, tenemos el botón stop o si queremos parar y ejecutar de nuevo el icono Relaunch



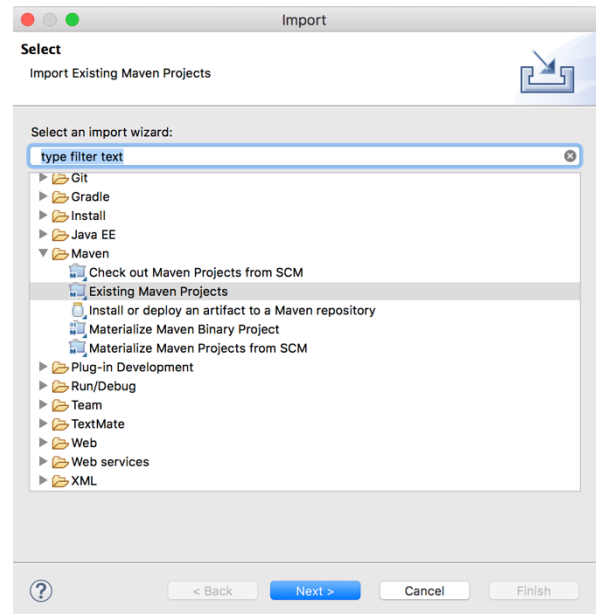
Antes de volver a compilar y ejecutar un proyecto, debemos pararlo para que no se acumulen las ejecuciones de todos los anteriores.

Para importar un proyecto ya creado

Basta con pinchar en File/import y elegir dentro de la carpeta Maven "Existing Maven Projects".

Nos dirá si queremos copiar el proyecto a importar en el espacio de trabajo y en este sentido, todo es igual a como se hace en Eclipse.

Se verá en entornos la forma de trabajar con Git.



Otros dos ejemplos de controladores

Puedes agregar estos métodos al ejemplo anterior:

```

/*
 * Ejemplo 2: Segundo controlador llamado welcome2
 * Atiende a la petición /saludo2 escrita en el @GetMapping
 *
 * Se puede ver cómo se "pasa o carga información" al
 * model dentro del método mediante el método addAttribute
 * como un objeto de la clase Persona, con valores de sus dos
 * atributos. Más adelante iremos viendo de dónde sacamos esa
 * "información" que
 * cambiar, como aquí la new Persona creada directamente en el método
 * addAttribute.
 */

@GetMapping("/saludo2")
public String welcome2(Model model) {

    model.addAttribute("persona", new Persona("Ángel", "Naranjo
    González"));
    return "SaludoPersonalizado";
}

/*
 * @GetMapping es una variante de requestMapping, más "nueva" que se
 * utiliza

```



```

    * como atajo, ya que basta con el nombre del recurso, mientras que
    * con @RequestMapping, en general, tenemos que ir diciendo el tipo de
petición
    * que se está atendiendo, es decir, necesita que le indiquemos el value
    * (nombre el recurso, por ejemplo, /saludo3) y el método que se usa para
la petición, en nuestro caso, tendríamos que escribir:
    * @RequestMapping (value="/saludo3", method=RequestMethod.GET)
    * (también existe RequestMethod.POST)
    * En general, se usa por simplificar el código
    * Se puede ver un ejemplo en:
    * https://www.arquitecturajava.com/spring-getmapping-postmapping-etc/
    *
    * Nosotros usaremos siempre GetMapping
    */

/*

/* Ejemplo 3: Tercer controlador llamado welcome3
* Atiende a la petición get "/saludo3"
*
* */

@GetMapping ("/saludo3")
public String welcome3(Model model) {

    model.addAttribute("saludo", "Hola Mundo");
    model.addAttribute("mensaje", "¡Se me está haciendo largo el
proyecto final!");
    model.addAttribute("url", "https:// triana.salesianos.edu");

    return "SaludoYEnlace";
}

```

Las páginas html de estos nuevos ejemplos se hacen igual a la primera llamada index, new File...:

Para welcome2:

SaludoPersonalizado.html

```

<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">

<head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1,
shrink-to-fit=no">

    <title>Saludo personalizado</title>
</head>

<body>

```

```
<h1>
  Hola <span th:text="${persona.nombre + ' ' + persona.apellidos}">
    amigo!</span>
</h1>

</body>

</html>
```

Para welcome3:

SaludoYEnlace.html

```
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">

<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1,
    shrink-to-fit=no">

  <title>Saludo y enlace</title>
</head>

<body>

  <h1 th:text="${saludo}"> </h1>
  <p th:text="${mensaje}"></p>
  <p>
    <a class="btn btn-primary btn-lg" target="_blank" th:href="${url}"
      role="button">Learn more &raquo;</a>
  </p>

</body>

</html>
```

3. Primeros pasos con Thymeleaf

A partir de ahora, empezaremos a estudiar las diferentes formas de usar Thymeleaf, en cuanto a qué componentes pintar y cómo hacerlo.

Utilizaremos algunos videotutoriales sueltos de internet y los propios tutoriales de Thymeleaf.

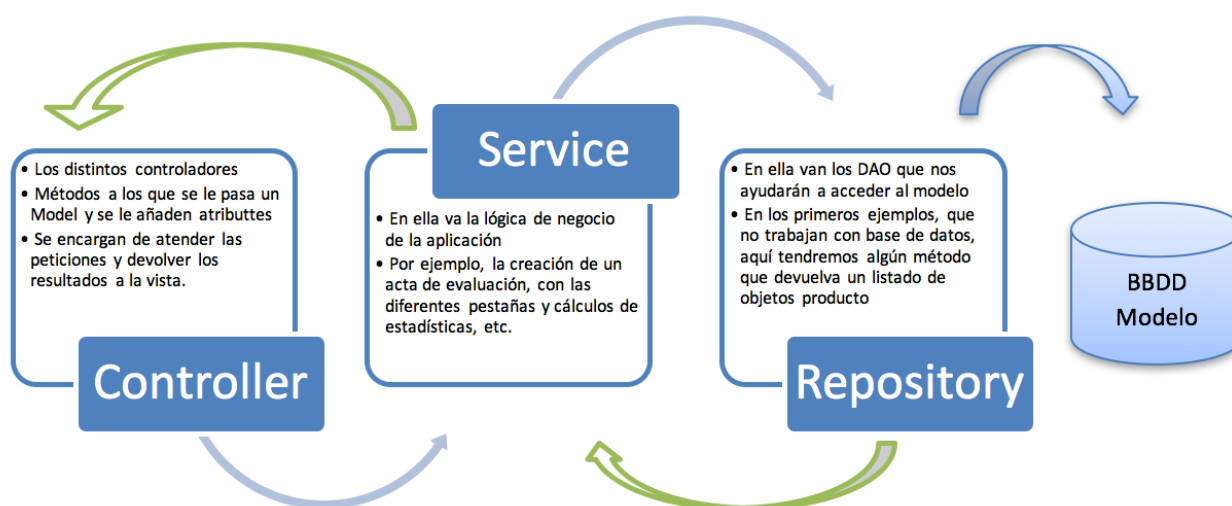
Para elementos más complejos con Thymeleaf y su integración en el proyecto iremos estudiando ejemplos concretos.

La estructura de carpetas de un proyecto acabado será más o menos como en el siguiente:

```

v karting [boot] [devtools]
v src/main/java
  > com.salesianostriana.dam.karting
  > com.salesianostriana.dam.karting.controller
  > com.salesianostriana.dam.karting.model
  > com.salesianostriana.dam.karting.repository
  > com.salesianostriana.dam.karting.security
  > com.salesianostriana.dam.karting.service
  > com.salesianostriana.dam.karting.service.baseservice
v src/main/resources
  > static
  > templates
  > application.properties
  > import.sql
v src/test/java
v JRE System Library [JavaSE-17]
v Maven Dependencies
v target/generated-sources/annotations
v target/generated-test-sources/test-annotations
v db
v src
v target
  mvnw
  mvnw.cmd
  pom.xml
  
```

Para tener una visión general de qué va en cada parte:



4. Configuración de Thymeleaf

Se puede hacer de varias formas, por ejemplo, mediante el uso de JavaConfig.

Nosotros configuraremos nuestras distintas opciones de Thymeleaf escribiendo lo necesario en el archivo `application.properties` en lugar de hacerlo usando `javaConfig`. Se irá viendo a lo largo de los distintos ejemplos en otros módulos.

